Event Time Dilation and the Variance Clock

Two clocks, day-weighted events, and a price-preserving vol drop

Vol-Fitter Technical Notes, No. 11

Vol-Fitter

June 28, 2026

Abstract

Variance does not accrue uniformly in calendar time: a scheduled event — earnings, an FOMC print, a court ruling — packs the variance of several normal days onto one. This note documents the Vol-Fitter's *variance clock*, which makes that explicit by carrying two times: calendar time t, and a dilated variance time τ in which an event day counts as several. Each event adds a number of *extra equivalent days*; the surface is fitted and quoted in weighted years $\tau = \tau_{\text{days}}/365$. The key invariant is price-preservation: total variance $w(k)$ is fixed by the observed option price and is clock-independent, so the working implied volatility $\sqrt{w/\tau}$ *drops* when an event is inserted before the expiry — exactly the overnight-vol-crush traders expect. An optional normalization rescales all days so the one-year variance budget is unchanged, making events *redistribute* variance within the year rather than add it. With no events the variance clock collapses to calendar time and every downstream fit is byte-identical to the plain pipeline. A worked example shows a 4-extra-day earnings event lifting short-dated implied vol and decaying away, with a working-vol drop of 36 vol bps at a fixed expiry just past the event.

Contents

1 Why two clocks

A diffusion model assumes variance accrues at a steady rate: total variance $w = \sigma^2 t$ is linear in calendar time. Real markets violate this around scheduled events, where a single day carries the variance of many. Forcing such a surface onto a single calendar clock produces the familiar pathology — a short-dated implied vol that looks "too high" and a term structure with a kink that a smooth model cannot represent without distorting the smile.

Heuristic

Separate the *calendar* (how many days until expiry) from the *variance budget* (how much variance those days carry). An earnings day is one calendar day but, say, five days' worth of variance. If we measure time in *variance days*, the diffusion looks steady again — and the "too high" short-dated implied vol is just the same total variance divided by a smaller calendar time. The event clock is the change of variable that restores $w = \sigma^2 \tau$ with a constant σ.

2 The variance clock

2.1 Day-weighted variance time

Each event carries a number N_e of *extra* equivalent days (an earnings day with $N_e = 4$ counts as 5 normal days of variance). The variance time to a calendar maturity t is

$$\tau_{\text{days}}(t) = t \cdot 365 + \sum_{t_e \le t} N_e, \qquad \tau(t) = \frac{\tau_{\text{days}}(t)}{365}, \tag{1}$$

counting only events at or before t. Figure 1 plots $\tau(t)$: it tracks the calendar line until the event, jumps by $N_e/365$ there, and runs parallel afterward. The fits and quotes live on τ; with no events, $\tau(t) = t$ exactly.

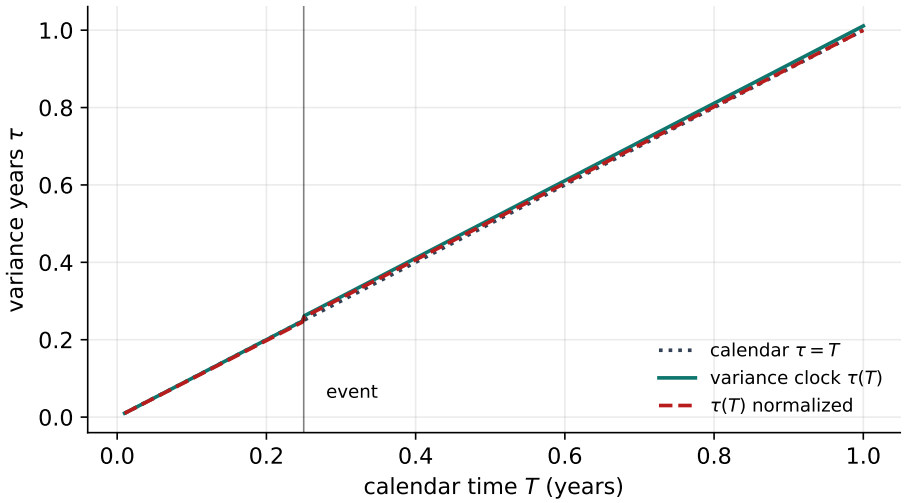

Figure 1: Variance time $\tau(t)$ for an event at $t_e = 0.25$ worth 4 extra days: it jumps at the event and runs parallel to calendar time thereafter. The normalized clock (dashed) rescales so the one-year budget is unchanged.

2.2 Price preservation

The crucial invariant is that the *total variance* $w(k)$ is fixed by the observed option price — it is the Black total variance that reprices the quote, and that is clock-independent. The variance clock only changes how we *report* it as a volatility:

$$\sigma_{\text{work}}(k) = \sqrt{\frac{w(k)}{\tau}}. \tag{2}$$

Inserting an event before the expiry raises τ at fixed w, so the working implied vol *drops*. This is the vol crush: the price is unchanged, but the *diffusive* volatility implied by it is lower once the event's variance is accounted for separately. Figure 2 shows the term-structure consequence: with the event, short-dated calendar implied vol is lifted (the event's variance compressed into few calendar days) and decays as the event becomes a smaller fraction of the maturity; without it, the term structure is flat.

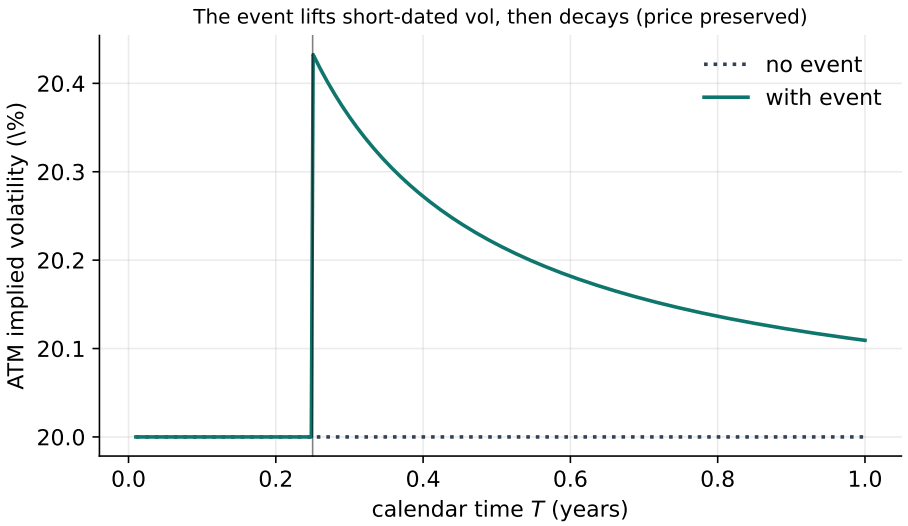

Figure 2: ATM implied-vol term structure with and without the event. The event lifts short-dated vol and decays away; the option *prices* are preserved, only the implied-vol reading changes.

2.3 Normalization

By default the cumulative weight exceeds the calendar-day count ($\tau > t$). With normalization on (an Options toggle), all days — event days included — are rescaled by a single factor so the one-year weight budget stays 365:

$$\tau_{\text{days}}(t) \;\longmapsto\; \tau_{\text{days}}(t) \cdot \frac{365}{365 + \sum_{t_e \le 1} N_e}. \tag{3}$$

Then the one-year variance time — and the one-year implied vol — are identical to a no-event year: events only *redistribute* variance within the year, they do not add it. Which convention to use is a desk choice: un-normalized treats the events as genuinely extra variance, normalized treats them as a known reallocation of a fixed annual budget.

3 How it threads the pipeline

The dual clock is prepared once per node: `weighted_variance_years` gives τ, and every fit (parametric and local-vol), the local-vol grid, the term structure and the option table consume τ in place of t. Total variance is interpolated *linearly in* τ between quoted nodes — flat forward variance per weighted-time unit — so the dense vol curve $\sqrt{w/\tau}$ stays bounded near zero and sensible past the last expiry. The term-structure view uses the shared calendar (not the per-node events) so the displayed curve is coherent across expiries. Changing the event calendar bumps the per-ticker events version, forcing a refit of only that name; with an empty calendar the whole pipeline is byte-identical to calendar time.

4 What is original here

Event time changes are not new in spirit (Bergomi and others dilate time around events), but the implementation here has two clean properties worth singling out. First, *price preservation by*

construction: the event clock never touches $w(k)$, only the volatility read-off (2), so adding an event can never change an option price — it cannot introduce arbitrage. Second, the *day-weighted* parametrization (1) makes the event size a single intuitive number (extra equivalent days) that a desk can quote and reason about, with an optional budget normalization (3) that cleanly separates "extra variance" from "reallocated variance". And the byte-identical fallback when the calendar is empty means the feature is strictly additive.

A Hyperparameter atlas

Table 1: Event-clock hyperparameters.

Knob	Default	Role
Surfaced (OptionsSettings)		
`events`	empty	List of `EventSpec`(t_e, N_e, label); N_e = extra equivalent days.
`normalizeEvents`	off	Rescale all days to a fixed 365 one-year budget (3).
Hidden		
`DAYS_PER_YEAR`	365	Calendar-to-variance day conversion.

Performance. The clock is a scalar sum per node, negligible. With no events $\tau = t$ and the entire downstream pipeline is byte-identical, so the feature adds zero cost when unused; changing the event calendar refits only the affected ticker (per-ticker events version).

References

[1] L. Bergomi. *Stochastic Volatility Modeling*. CRC Press, 2016.

[2] J. Gatheral. *The Volatility Surface*. Wiley, 2006.